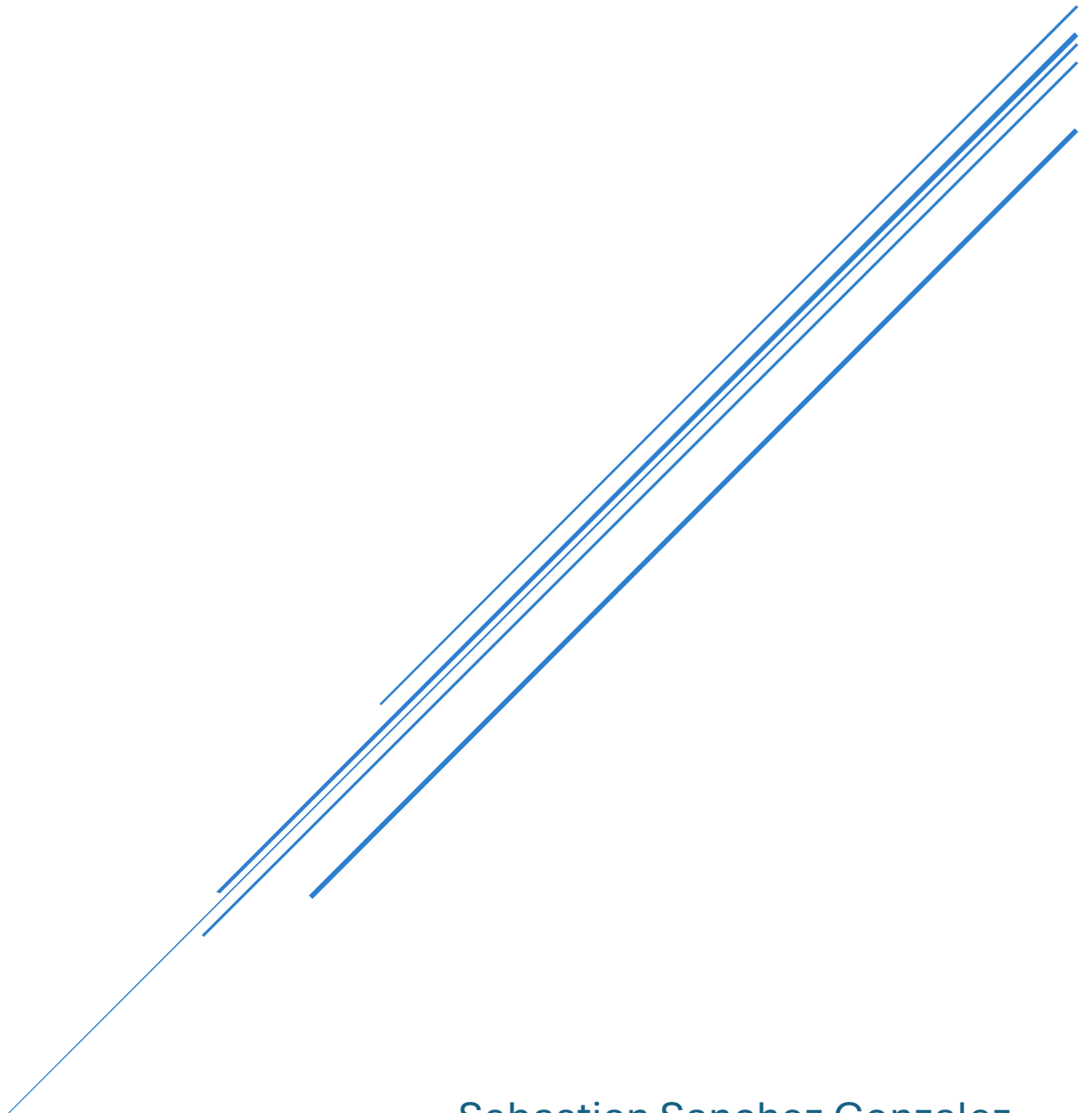


TP3 – DOCUMENT WORD

Application Web



Sebastian Sanchez Gonzalez
Maxime Maindron

Table des matières

Qu'avez-vous appris sur l'analyse d'un système ?	2
À quoi sert le diagramme entité-association dans le cadre de votre projet ?	3
Quels ont été les difficultés rencontrées dans le cadre de ce projet ?	4

Qu'avez-vous appris sur l'analyse d'un système ?

Dans le cadre de ce projet, j'ai appris que l'analyse d'un système repose sur plusieurs éléments fondamentaux :

1. **L'identification des entités** : Avant d'écrire une seule ligne de code, il faut d'abord repérer les objets du monde réel que le système doit représenter. Dans notre cas, cela correspond à des entités comme Fabricant, Console, Jeu, Genre et Exemple.
2. **Les relations entre les entités** : Une fois les entités identifiées, il faut comprendre comment elles interagissent entre elles. Par exemple, un fabricant produit des consoles, et une console accueille des jeux. Ces relations se traduisent par des clés étrangères dans la base de données.
3. **Les attributs et leur nature** : Chaque entité possède des attributs qui la décrivent. L'analyse distingue les attributs obligatoires (notés *) des attributs optionnels (notés o), comme année_edition ou prix_achat. Cette distinction influence directement les contraintes NOT NULL dans le script SQL.
4. **Les cardinalités (multiplicités)** : L'analyse précise combien d'occurrences d'une entité peuvent être liées à une autre. Par exemple, un fabricant peut produire plusieurs consoles (1 à plusieurs), mais une console n'appartient qu'à un seul fabricant. Ces règles définissent la structure relationnelle de la base.
5. **La séparation des responsabilités** : L'analyse m'a permis de comprendre comment diviser un système complexe en couches distinctes : la base de données (Oracle/ORDS), l'API REST (les endpoints), et l'interface Web (HTML/CSS/JS). Chaque couche a un rôle précis et communique avec les autres de façon structurée.

À quoi sert le diagramme entité-association dans le cadre de votre projet ?

Le diagramme entité-association a servi à plusieurs intervenants dans notre projet :

D'abord, il nous a servi à nous-mêmes, car il constitue le plan de départ avant de créer quoi que ce soit. En visualisant les entités (Fabricant, Console, Jeu, Genre, Exemplaire) et leurs relations, on a pu écrire le script SQL de création des tables de façon cohérente, en respectant l'ordre des dépendances (par exemple, créer Fabricant avant Console, puisque Console référence Fabricant).

Ensuite, il sert aux enseignants et évaluateurs, qui peuvent vérifier rapidement si la structure de la base de données est bien pensée, si les relations sont logiques, et si les contraintes d'intégrité (clés primaires, clés étrangères) ont été correctement identifiées.

Enfin, il pourrait servir à tout futur développeur qui rejoindrait le projet : le diagramme permet de comprendre la logique des données sans avoir à lire tout le code SQL ou le JavaScript. C'est un outil de communication universel entre les personnes qui conçoivent un système et celles qui vont le maintenir ou l'utiliser.

Quels ont été les difficultés rencontrées dans le cadre de ce projet ?

La principale difficulté que j'ai rencontrée dans ce projet concerne le code JavaScript, plus particulièrement la logique de chargement dynamique des données.

Au départ, je m'étais décidé de créer une page HTML distincte pour chaque fabricant et pour chaque jeu. Cependant, j'ai remarqué assez rapidement que cette approche était non seulement inefficace, mais aussi incompatible avec une vraie application Web dynamique connectée à une base de données. J'ai donc dû comprendre et mettre en œuvre une approche différente : utiliser une seule page HTML réutilisable, et laisser JavaScript interroger la base de données via l'API ORDS pour déterminer quelles informations afficher selon le contexte.

Concrètement, cela signifiait passer un paramètre dans l'URL (par exemple, ?fabricant=Nintendo ou ?id_exemplaire=3), le lire avec URLSearchParams, puis effectuer les bons appels fetch() pour récupérer uniquement les données pertinentes et les injecter dynamiquement dans le DOM. Cette logique m'a demandé plusieurs recherches pour en comprendre le fonctionnement et l'implémenter correctement.

Une autre difficulté a été la création des boutons et des formulaires dans le site. Il ne suffisait pas de les écrire en HTML : il fallait également les relier à des fonctions JavaScript pour qu'ils soient réellement fonctionnels. Par exemple, les boutons d'ajout devaient récupérer les valeurs saisies dans les champs du formulaire, les valider, puis envoyer les données à la base de données via un appel POST à l'API. De même, les boutons de suppression et d'achat devaient déclencher les bons appels DELETE ou PUT et mettre à jour l'affichage immédiatement, sans recharger la page. Coordonner tout cela correctement a nécessité plusieurs ajustements et tests avant d'obtenir le comportement souhaité. J'ai aussi dû effectuer plusieurs recherches pour m'assurer de bien comprendre leur fonctionnement et de les implémenter correctement.

Cette difficulté m'a permis de mieux comprendre comment fonctionne réellement une application Web moderne : ce n'est pas le nombre de pages qui change, mais bien le contenu que JavaScript y place, en fonction des données reçues de l'API.